

# 铺砖问题求解

刘懿仁

57123130

2025 年 10 月 9 日

## 摘要

编写一个程序，在 $16 \times 16$ 的方格中，有任意一个空格，输出在剩余格子中铺“L”型地砖的方案。

## 1 问题分析

该问题适合采用分治策略，先将 $16 \times 16$ 的方格分为四个 $8 \times 8$ 的方格，将一块L型地砖铺在中心位置，缺口朝向有空格的方格方向，这样问题就变成了在4个有空格的 $8 \times 8$ 方格中铺设L型地砖，以此递归。

## 2 问题的求解

先将 $16 \times 16$ 的方格从中心分为四个 $8 \times 8$ 的方格，将一块L型地砖铺在中心位置，该地砖的缺口朝向有空格的 $8 \times 8$ 方格的方向，这样原来的空格的范围就缩小到了一个 $8 \times 8$ 的地砖，然后可以把剩下三个 $8 \times 8$ 方格中已经铺好的地砖看作一个空格，这样问题就变成了在4个有一个空格的 $8 \times 8$ 方格中铺设L型地砖的问题，即四个与原来相同但规模较小的问题，以此递归。  
base情况：按照上述思路递归进行下去，直到最后问题最后递归为在一

个2×2方格中铺一个L型地砖，这时只需要把地砖铺在除了空格之外的其他方格中即可，这是该问题的基础情况。

```
1  #include <iostream>
2  using namespace std;
3
4  #define MAX 1025
5  int k; // whole size
6  int x, y; // location of the empty block
7  int board[MAX][MAX];
8  int tile = 1; // initialize the block as 1
9
10 void ChessBoard(int tr, int tc, int dr, int dc, int
    size) { // Define a chessboard segmentation
    function with parameters including the starting
    row and column (tr, tc), special grid positions
    (dr, dc), and size
11     if (size == 1) return; // return when size = 1
12
13     int t = tile++; // Assign tile number to t and
    increment tile number
14     int s = size / 2;
15
16     if (dr < tr + s && dc < tc + s) { // the empty
    block is in the upperleft childblock
17         ChessBoard(tr, tc, dr, dc, s);
18     }
19     else {
20         board[tr + s - 1][tc + s - 1] = t; //
    record the location of empty block
21         ChessBoard(tr, tc, tr + s - 1, tc + s - 1,
    s); // recursive processing upperleft
```

```

22         childblock
23     }
24     if (dr < tr + s && dc >= tc + s) { // the
25         empty block is in the upperright childblock
26         ChessBoard(tr, tc + s, dr, dc, s);
27     }
28     else {
29         board[tr + s - 1][tc + s] = t;
30         ChessBoard(tr, tc + s, tr + s - 1, tc + s,
31             s); // recursive processing upperright
32             childblock
33     }
34     if (dr >= tr + s && dc < tc + s) { // the
35         empty block is in the lowerleft childblock
36         ChessBoard(tr + s, tc, dr, dc, s);
37     }
38     else {
39         board[tr + s][tc + s - 1] = t;
40         ChessBoard(tr + s, tc, tr + s, tc + s - 1,
41             s); // recursive processing lowerleft
42             childblock
43     }
44     if (dr >= tr + s && dc >= tc + s) { // the
45         empty block is in the lowerright childblock
46         ChessBoard(tr + s, tc + s, dr, dc, s);
47     }
48     else {
49         board[tr + s][tc + s] = t;

```

```

45         ChessBoard(tr + s, tc + s, tr + s, tc + s,
46                   s); // recursive processing lowerright
47         childblock
48     }
49 }
50
51 int main() {
52     k = 4; // the size of whole block is (2^4)^2
53     srand((unsigned)time(NULL));
54     x = rand() % 16;
55     y = rand() % 16;
56     int size = 1 << k;
57     ChessBoard(0, 0, x, y, size);
58     for (int i = 0; i < size; i++) {
59         for (int j = 0; j < size; j++) {
60             printf("%4d", board[i][j]);
61         }
62         printf("\n");
63     }
64     return 0;
65 }

```

### 3 代码运行结果

如图，红色圆圈（0）中为空缺的地砖，红线为第一次分治的区域，将 $16 \times 16$ 的方格分为了四个 $8 \times 8$ 的方格，可以看出，1号L型地砖的缺口对应左上角的 $8 \times 8$ 方格，也就是缺口所在的区域。

```
Microsoft Visual Studio 调试器
4  4  5  5  9  9 10 10 25 25 26 26 30 30 31 31
4  3  3  5  9  8  8 10 25 24 24 26 30 29 29 31
6  3  7  7 11 11 8 12 27 24 28 28 32 32 29 33
6  6  0  7  2 11 12 12 27 27 28 23 23 32 33 33
14 14 15 2  2 19 20 20 35 35 36 36 23 40 41 41
14 13 15 15 19 19 18 20 35 34 34 36 40 40 39 41
16 13 13 17 21 18 18 22 37 37 34 38 42 39 39 43
16 16 17 17 21 21 22 22 1 37 38 38 42 42 43 43
46 46 47 47 51 51 52 1  1 67 68 68 72 72 73 73
46 45 45 47 51 50 52 52 67 67 66 68 72 71 71 73
48 45 49 49 53 50 50 54 69 66 66 70 74 74 71 75
48 48 49 44 53 53 54 54 69 69 70 70 65 74 75 75
56 56 57 44 44 61 62 62 77 77 78 65 65 82 83 83
56 55 57 57 61 61 60 62 77 76 78 78 82 82 81 83
58 55 55 59 63 60 60 64 79 76 76 80 84 81 81 85
58 58 59 59 63 63 64 64 79 79 80 80 84 84 85 85

D:\cpp\Algorithm00\x64\Debug\Algorithm00.exe (进程 46972)已退出, 代码为 0 (0x0)。
按任意键关闭此窗口 . . .
```

图 1: execution result